Sequential queue

Implement a specialized .NET collection similar to a queue with the difference that the output is not first-in-first-out and instead the order of the output is determined by a sequential number that is given to the input.

- The queue is used asynchronously from multiple threads.
- The queue stores instances of classes of a single given type.
- The input is a sequence number of long integer type and the object to be stored.
- The sequence number starts with 1 and is guaranteed sequential. The consumer of the queue cannot skip a sequence number.
- When adding items to the queue the sequence numbers (and associated objects) do not have to be provided in order, i.e. 4, 2, 3, 1, 5… is a valid input.
- The output of the queue is always in the correct order, i.e. 1, 2, 3, 4, 5
- The Dequeue operations return the sequence number and the stored object.
- If the implementation needs temporary memory storage, the consumer should be able to limit the maximum used storage.
- There are two methods each to enqueue and dequeue an entry:
    a. Enqueue and wait if the enqueue operation is temporarily not possible. Dequeue and wait if no entry is available.
    b. Try to perform the enqueue or dequeue operation. Return immediately and indicate to the calling code if the operation could be performed.

Example

Enqueue operation can be out of order:

(4, object4)
(2, object2)
(3, object3)
(1, object1)
(5, object5)

Dequeue operation always is in order:

(1, object1)
(2, object2)
(3, object3)
(4, object4)
(5, object5)

Implement the queue in a way so that it can be used reliably in Enterprise Software with confidence.

Implement in C# for .NET

Confidential